Introduction to Bioinformatics

Short Read Alignment

Ali Sharifi-Zarchi Somayyeh Koohi

CE Department
Sharif University of Technology

Fall 2025-26

Presentation Overview

The Read Alignment Problem

- DNA sequencing $\rightarrow$ millions of short reads (50–150 bp)
- We have a long reference genome (e.g., human: 3 billion bp)

- Goal: For each read, find where (if anywhere) it came from in the reference

$$\Rightarrow \text{Massive approximate string matching}$$

Real-World Challenges

- Reference size: bacteria $\sim$5Mbp, human $\sim$3Gbp
- Number of reads: 10^6 – 10^9
- Reads are short and noisy (sequencing errors, SNPs, indels)
- Genomes are highly repetitive
- Must finish in hours on limited RAM/CPU

What Do Modern Aligners Need?

Key Requirements

- High sensitivity (find true origin even with a few errors)
- Very high speed & scalability
- Low memory usage (index must fit in RAM)
- Reference indexed once, queried millions of times

Pairwise Sequence Alignment

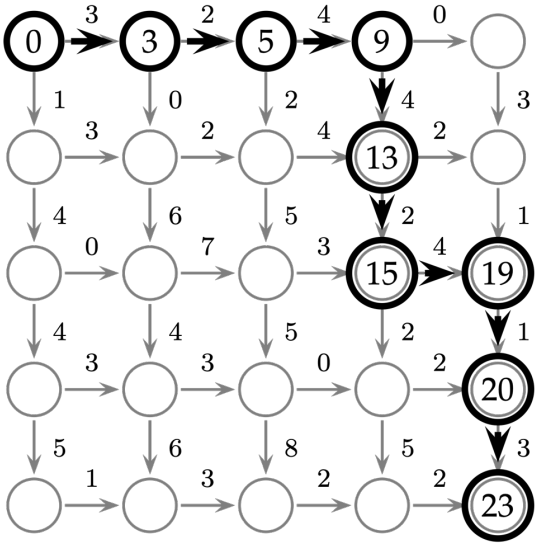
Core idea

Compare a short read (pattern) against every possible position in the reference, allowing mismatches and gaps.

Two classic formulations:

- Global alignment (Needleman-Wunsch) - entire read
- Local alignment (Smith-Waterman) - best substring

The Manhattan Tourist Problem (warm-up)



Find the maximum-weight path from top-left to bottom-right

⇓

Recurrence:

$$s_{i,j} = \max \begin{cases} s_{i-1,j} + w(\text{vertical}) \\ s_{i,j-1} + w(\text{horizontal}) \end{cases}$$

General DP Strategy

1. Turn the problem into a path-finding problem in a grid
2. Define a recurrence: value of cell (i, j) depends only on a few previous cells
3. Fill table row-by-row (or column-by-column)
4. Trace back to recover the actual alignment

Global, Local, Multiple

Target Sequence

```
5' ACTACTAGATTACTTACGGATCAGGTACTTTAGAGGCTTGCAACCA 3'
              |||| |||||| ||||||||||||||||
```

Query Sequence `5' TACTCACGGATGAGGTACTTTAGAGGC 3'`

Global Alignment

Target Sequence

```
5' ACTACTAGATTACTTACGGATCAGGTACTTTAGAGGCTTGCAACCA 3'
   ||||||||||||    |||||||  |||||||||||||| |||||||
5' ACTACTAGATT----ACGGATC--GTACTTTAGAGGCTAGCAACCA 3'
```

Query Sequence

Multiple Sequence Alignment (MSA)

Figure: Comparison between the kinds of alignment

Needleman-Wunsch (1970) - Global Alignment

Recurrence (simplified scoring: match +1, mismatch/gap -1):

$$F(i,j) = \max \begin{cases} F(i-1,j-1) + s(A_i, B_j) & \text{diagonal (match/mismatch)} \\ F(i-1,j) - d & \text{vertical (gap in B)} \\ F(i,j-1) - d & \text{horizontal (gap in A)} \end{cases}$$

Needleman-Wunsch Example

Scoring: match = +1, mismatch = -1, gap = -1

	–	A	C	T	G	A	T	C	A
–	**0**	-1	-2	-3	-4	-5	-6	-7	-8
A	-1	**1**	**0**	-1	-2	-3	-4	-5	-6
T	-2	0	0	**1**	0	-1	-2	-3	-4
G	-3	-1	-1	0	**2**	1	0	-1	-2
A	-4	-2	-2	-1	1	**3**	2	1	0
T	-5	-3	-3	-1	0	2	**4**	3	2
C	-6	-4	-3	-2	-1	1	3	**5**	4
A	-7	-5	-4	-3	-2	0	2	4	**6**

Final score = 6 $\Rightarrow$ very good global alignment

- **Red 0 = starting point (empty alignment)**

Smith-Waterman (1981) - Local Alignment

Same three choices, but we are allowed to start fresh:

$$H(i,j) = \max \begin{cases} 0 \\ H(i-1,j-1) + s(A_i, B_j) \\ H(i-1,j) - d \\ H(i,j-1) - d \end{cases}$$

- 0 means ”start a new alignment here”
- Optimal local alignment = maximum value in the entire matrix
- Trace back from that maximum until reaching 0

Global vs. Local

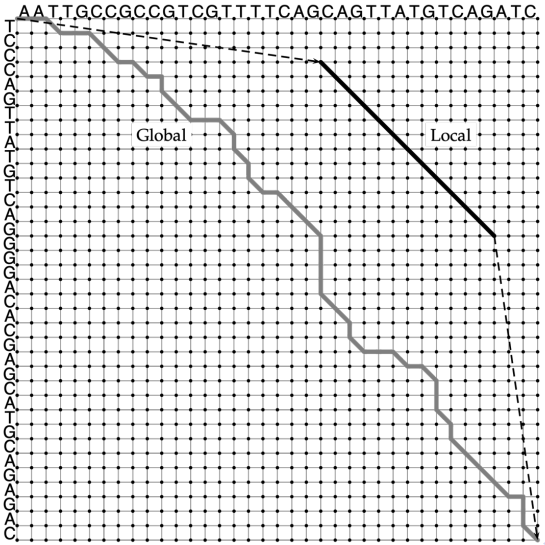
```
--T--CC-C-AGT--TATGT-CAGGGGACACG--A-GCATGCAGA-GAC
  |  || |  ||  | | | |||    || |  | |  | ||||   |
AATTGCCGCC-GTCGT-T-TTCAG----CA-GTTATG--T-CAGAT--C
```

Figure: Global alignment example

```
                    tccCAGTTATGTCAGgggacacgagcatgcagagac
                       ||||||||||||
aattgccgccgtcgttttcagCAGTTATGTCAGatc
```

Figure: Local alignment example

Global vs. Local



Affine Gap Penalties

Real gaps are usually long $\rightarrow$ opening a gap is more expensive than extending it.
$\Rightarrow$ Use three matrices (Gotoh 1982):

- $M(i, j)$ - best alignment ending with match/mismatch
- $I_x(i, j)$ - best ending with gap in sequence x
- $I_y(i, j)$ - best ending with gap in sequence y

Widely used in BLAST, BWA, Bowtie2, etc.

Complexity Problem

- Needleman-Wunsch / Smith-Waterman: $O(mn)$ time and space
- m = read length (100), n = reference length (3×10^9)
- Trying every position $\rightarrow$ 3×10^{11} cell computations per read
- Even at 10^9 cells/sec $\rightarrow$ hours per read $\rightarrow$ impossible

Conclusion: We cannot afford full DP for every read and every position.

Triple Sequence Alignment

Example

```
--T--CC-C-AGT--TATGT-CAGGGGACACG--A-GCATGCAGA-GAC
  |   || |   ||   | | | |||     || |   | |   | ||||    |
AATTGCCGCC-GTCGT-T-TTCAG----CA-GTTATG--T-CAGAT--C
 |||||  |    X|||| |           || XXX|||   | ||| |   |
-ATTGC-G--ATTCGTAT------GGGACA-TGGATGCATGCAG-TGAC
```

Figure: Triple alignment example

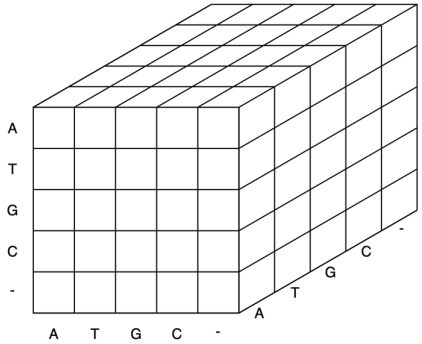
Triple Sequence Alignment

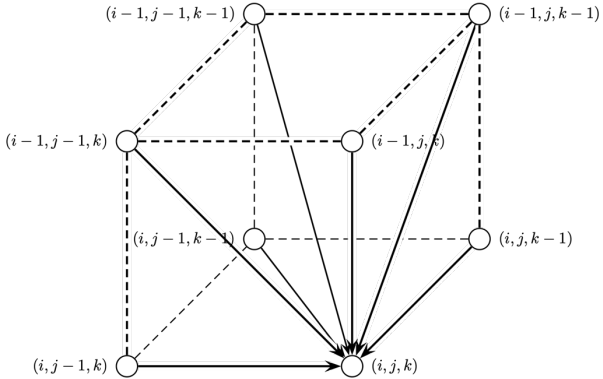
Policy

$$s_{i,j,k} = \max \begin{cases} s_{i-1,j,k} & +\delta(v_i, -, -) \\ s_{i,j-1,k} & +\delta(-, w_j, -) \\ s_{i,j,k-1} & +\delta(-, -, u_k) \\ s_{i-1,j-1,k} & +\delta(v_i, w_j, -) \\ s_{i-1,j,k-1} & +\delta(v_i, -, u_k) \\ s_{i,j-1,k-1} & +\delta(-, w_j, u_k) \\ s_{i-1,j-1,k-1} & +\delta(v_i, w_j, u_k) \end{cases}$$

Triple Sequence Alignment

3-dimensional DP table





Summary & Transition

What we learned

- Pairwise alignment $\rightarrow$ solved elegantly with DP
- Global (Needleman-Wunsch) and local (Smith-Waterman)
- Affine gaps make scoring biologically realistic

But

Pure DP is far too slow for millions of reads against huge genomes.

Why the Burrows-Wheeler Transform (BWT)?

Real-world impact

- Illumina NovaSeq: $>$ 3 terabases of DNA per day
- Human genome index must fit in RAM and answer queries in microseconds

BWT powers nearly every modern short-read aligner:

- Bowtie / Bowtie2 BWA (MEM & ALN) minimap2 SNAP GEM ...

$\Rightarrow$ The single most important data structure in bioinformatics today

BWT Construction – “panama$” Example

`S = panama$` ($ is smallest sentinel)

#	Rotation	Sorted rotation	BWT
0	panama$	$panama	a
1	anama$p	a$panam	m
2	nama$pa	ama$pan	n
3	ama$pan	anama$p	p
4	ma$pana	ma$pana	a
5	a$panam	nama$pa	a
6	$panama	panama$	$

$\Rightarrow$ BWT = amnpaa$

The Full BWT Matrix with Ranks (LF Mapping)

`BWT = amnpaa$`

i	F (sorted)	L[i]	Rank in L	Rank in F $\rightarrow$ LF(i)
0	$panama	a	a^1	$a^1 \rightarrow 1$
1	a$panam	m	m^1	$m^1 \rightarrow 4$
2	ama$pan	n	n^1	$n^1 \rightarrow 5$
3	anama$p	p	p^1	$p^1 \rightarrow 6$
4	ma$pana	a	a^2	$a^2 \rightarrow 2$
5	nama$pa	a	a^3	$a^3 \rightarrow 3$
6	panama$	$	$\$^1$	$\$^1 \rightarrow 0$

alertLF rule: *k*-th *c* in L $\rightarrow$ *k*-th *c* in F

FM-Index: What We Store in Practice

C[c] = count of chars $<$ c

- C[$]=0, C[a]=1, C[m]=4, C[n]=5, C[p]=6

LF(i) = C[BWT[i]] + Occ(BWT[i], i-1)

$\Rightarrow$ Human genome index $\approx$ 3–5 GB

Occ(c,i) = occurrences of *c* in BWT[0..i]

Constructing BWT from Suffix Array

Python Code

```
def compute_bwt(s, sa):
bwt = []
for p in sa:
if p == 0:
bwt.append('$')
else:
bwt.append(s[p-1])
return ''.join(bwt)
```

Caveat

Constructing the full suffix array still costs $O(n \log n)$ time and $O(n)$ extra space $\Rightarrow$ this removes the memory benefits of the BWT!

The Crucial Property: Last-to-First (LF) Mapping

Index i	$L[i]$ (BWT)	$F[i]$ (first column)
0	t	$
1	t	a
2	t	a
3	t	a
4	$	c
5	a	t
6	a	t
7	a	t
8	c	t

LF mapping rule: If $L[i] = c$ is the k-th occurrence of c in the BWT (column L), then $\text{LF}(i)$ is the row where the k-th occurrence of c appears in column F.

$\Rightarrow$ Following LF repeatedly walks backwards through the original string!

Decoding the Original String using LF Mapping

Python code

```
def decode_bwt(bwt, LF):
  n = len(bwt)
  r = 0                # start at row containing $
  s = ['$']
  while len(s) < n:
    s.append(bwt[r])     # prepend the previous character
    r = LF[r]          # go to previous position
  return ''.join(reversed(s))
```

Result: Starting from the $ row and repeatedly applying LF reconstructs the original text in reverse.

LF Mapping in Practice – C and Occ arrays

We don't store the full LF array (would cost $O(n)$ space). Instead we use two small tables:

- $C[c]$ = number of characters strictly smaller than c in the text
- $\mathrm{Occ}(c, i)$ = number of occurrences of c in $\mathrm{BWT}[0..i]$

Then:

$$\mathrm{LF}(i) = C[\mathrm{BWT}[i]] + \mathrm{Occ}(\mathrm{BWT}[i], i-1)$$

$\Rightarrow$ This is the core of the FM-index (Ferragina–Manzini 2000).

FM-Index = Compressed Full-Text Index

The FM-index supports backward searching of patterns in $O(m)$ time (often sub-linear with sampling):

1. Start with interval $[0, n)$
2. For each character c of the pattern from right to left:

$$i' = C[c] + \text{Occ}(c, i-1)$$
$$j' = C[c] + \text{Occ}(c, j) - 1$$

3. If $i' \leq j' \Rightarrow$ continue, else pattern not present

This is exactly how Bowtie, BWA-MEM, etc. find seed locations fast.

Backward Search – Find “ana” in “panama$”

Start: [0,7) Pattern: a $\rightarrow$ n $\rightarrow$ a (right to left)

Step	Char	Calculation	Interval
0	–	–	[0,7)
1	a	1+0=1 … 1+3=4	alert[1,4)
2	n	5+0=5 … 5+1=6	alert[5,6)
3	a	1+2=3 … 1+3=4	alert[3,4) Success

“ana” occurs once, starting at position 2

Backward Search – Visualizing Step 1 (final “a”)

Current interval [0,7) $\rightarrow$ looking for preceding a

i	BWT[i]	F row
0	a	$panama
1	m	a$panam $\rightarrow$ a^1
2	n	ama$pan $\rightarrow$ a^2
3	p	anama$p $\rightarrow$ a^3
4	a	ma$pana
5	a	nama$pa
6	$	panama$

3× a $\rightarrow$ new interval [1,4)

Backward Search – Step 2: Process “n”

Current interval: [1,4) $\rightarrow$ rows 1–3 only
We are now looking for preceding character n

Only row 2 has BWT[2] = n in current range $\rightarrow$ its preceding char (via LF) is n

The 1st ‘n’ in F is at row 5 $\rightarrow$ new interval becomes [5,6)

Backward Search – Step 3: Process first “a” again

Current interval: [5,6) $\rightarrow$ only row 5
BWT[5] = a $\rightarrow$ matches our pattern ‘a’
$\rightarrow$ maps to row 3 (3rd ‘a’ in F)

Final interval [3,4) $\rightarrow$ exact match found!

Conclusion: “ana” occurs once, starting at position 2 in “panama$”

Summary – Why BWT is Perfect for Short-Read Alignment

- Reversible transformation
- Highly compressible (runs of identical characters)
- Enables exact backward search in $O(m)$ time using only $O(n)$ bits
- With sampling $\Rightarrow$ locate occurrences in constant time
- Whole human genome index fits in $\leq$ 4 GB RAM
- Used by virtually every modern short-read aligner (Bowtie, BWA, etc.)

Next step: Seed-and-extend using these exact matches $\rightarrow$ fast, sensitive alignment.

Why is Seed & Extend Necessary?

- A full exact BWT search works well only when the read has **no errors**.
- Real sequencing data contains:
  - Substitutions
  - Insertions and deletions
  - PCR/sequencing errors
- Supporting errors directly inside the BWT search is expensive.
- **Solution:** Modern aligners index the genome but search only short exact (or near-exact) **seeds**.

Key Idea

Instead of aligning the whole read, find a reliable seed quickly, and then extend it locally to reconstruct the full alignment.

- This reduces the search space by orders of magnitude.
- Makes BWT-based alignment practical, fast, and scalable.

Full Alignment vs. Seed & Extend

Full Alignment (DP or full BWT search)

- Attempts to align the entire read at once.
- Expensive in both time and memory.
- Error tolerance requires exploring many paths.
- Not scalable for billions of bases.

Seed & Extend

- Search only short seeds in the index.
- Very fast FM-index exact matching on seeds.
- Extension uses a small DP or local BWT search.
- Scales to whole human genome efficiently.

Conclusion

Seed & Extend combines:

- **Speed** from FM-index seed lookup, and
- **Accuracy** from local DP extension.

This is why all modern short-read aligners use it.

Contributors

Sina Beyrami

Sina Daneshgar

References

- **Istrail, S., Pevzner, P.** *An Introduction to Bioinformatics Algorithms*.
- **Needleman and Wunsch** (1970) *A general method applicable to the search for similarities in the amino acid sequence of two proteins*
- **Smith and Waterman** (1981) *Identification of common molecular subsequences*
- **Ugene** Alignment of Short Reads
- **Wikipedia**
- **Sven Rahmann** (2021). *The Burrows-Wheeler Transform (BWT) – Algorithms for Sequence Analysis*.
- **Burrows M., Wheeler D. J.** (1994). *A block-sorting lossless data compression algorithm*.
- **Ferragina P., Manzini G.** (2000). *Opportunistic data structures with applications*.
- **Simpson J. T., Durbin R.** (2010). *Efficient construction of an assembly string graph using the FM-index*.
- **Langmead B., Trapnell C., Pop M., Salzberg S. L.** (2009). *Ultrafast and memory-efficient alignment of short DNA sequences to the human genome*.
- **Li H., Durbin R.** (2009). *Fast and accurate short read alignment with Burrows-Wheeler transform*.